

Mesterséges Intelligencia alkalmazása a Connect4 játékban

1. A feladat célja

A projekt célja egy korábban elkészített Connect4 játék továbbfejlesztése volt úgy, hogy a játék tartalmazzon egy mesterséges intelligencián alapuló döntéshozó modult. A követelmények alapján a megoldásnak állapottér-gráf reprezentációra kellett épülnie, valamint egy tanult keresőalgoritmust (minimax, alfa-béta vágással) kellett alkalmaznia.

A fejlesztés során a ChatGPT mesterséges intelligenciát használtam segítségként a tervezéshez, implementációhoz és hibakereséshez.

2. Az állapottér modell kialakítása

Az AI javaslatára létrehoztam egy külön `GameState` osztályt, amely a játék aktuális állapotát reprezentálja.

Megvalósítás helye:

- `GameState` osztály

A `GameState` tartalmazza:

- a játék tábla (`char[][] board`),
- az aktuális játékos (`isMaxPlayerTurn`),
- a tábla másolására szolgáló metódust (`copyBoard()`).

A mély másolás biztosítja, hogy a minimax algoritmus során az állapotok egymástól függetlenek maradjanak.

3. Terminális állapot kezelése

Az `isTerminal()` metódus eldönti, hogy egy állapot végállapot-e.

Megvalósítás helye:

- `GameState` osztály

Feltételek:

- valamelyik játékos nyert,

- vagy nincs több lehetséges lépés.
-

4. Győzelemellenőrzés

A győzelemellenőrzés több irányban történik:

- vízszintes,
- függőleges,
- átlós (két irányban).

Megvalósítás helye:

- GameState osztály → minimaxhoz szükséges ellenőrzés
 - Game osztály → aktuális játékban győzelem felismerése (checkWin, checkDirection)
-

5. Lépésgenerálás

A makeMove(int col) metódus létrehozza a következő állapotot.

Megvalósítás helye:

- GameState osztály

Funkció:

- korong elhelyezése,
 - új állapot létrehozása,
 - játékosváltás kezelése.
-

6. Értékelő függvény (heurisztika)

Az evaluate() függvény pontozza az állapotokat.

Megvalósítás helye:

- GameState osztály

Segédmetódusok:

- evaluateWindows()
- evaluateWindow()

Az értékelés 4-es minták alapján történik, pozitív (AI) és negatív (ellenfél) pontozással.

7. Minimax algoritmus

A minimax algoritmus kiválasztja a legjobb lépést.

Megvalósítás helye:

- `MinimaxAI` osztály

Fő metódusok:

- `findBestMove()`
 - `minimax()`
-

8. Alfa–béta vágás

A minimax algoritmus optimalizálása alfa–béta vágással történt.

Megvalósítás helye:

- `MinimaxAI` osztály (a `minimax()` metóduson belül)

Cél:

- felesleges ágak levágása,
 - futási idő csökkentése.
-

9. AI integrálása a játékba

A minimax algoritmus a `Game` osztályban került integrálásra.

Megvalósítás helye:

- `Game` osztály

Kapcsolódó metódusok:

- `computerMove()` → `minimax` meghívása
 - `convertBoardForAI()` → tábla átalakítása a `GameState` számára
-

10. Kezdőállás betöltése fájlból

A játék képes kezdőállást betölteni fájlból.

Megvalósítás helye:

- `Board` osztály $\rightarrow$ `loadFromFile()`
- `Game` osztály $\rightarrow$ felhasználói választás kezelése (`startGame()`)

A fájlban:

- `.` jelöli az üres mezőt,
 - `x` és `o` a játékosokat.
-

11. Játéktábla megjelenítés és színezés

A tábla konzolos megjelenítése színezéssel történik.

Megvalósítás helye:

- `Board` osztály $\rightarrow$ `printBoard()`

Színezés:

- `x` (AI) $\rightarrow$ piros
 - `o` (játékos) $\rightarrow$ sárga
-

12. AI-val való együttműködés

Az AI-t az alábbi területeken használtam:

- architektúra tervezése (`GameState`, `MinimaxAI`),
- algoritmus implementáció,
- hibakeresés,
- tesztelés,
- dokumentáció készítése.

A fejlesztés lépésenként történt, az AI segítségével.

13. Összegzés

A projekt során egy teljes értékű mesterséges intelligencia került integrálásra a Connect4 játékba.

A rendszer:

- állapottér-alapú,
- minimax algoritmust használ,
- alfa–béta vágással optimalizált,
- képes stratégiai döntések meghozatalára.

A megoldás megfelel a tantárgyi követelményeknek, és bemutatja a keresőalgoritmusok gyakorlati alkalmazását.